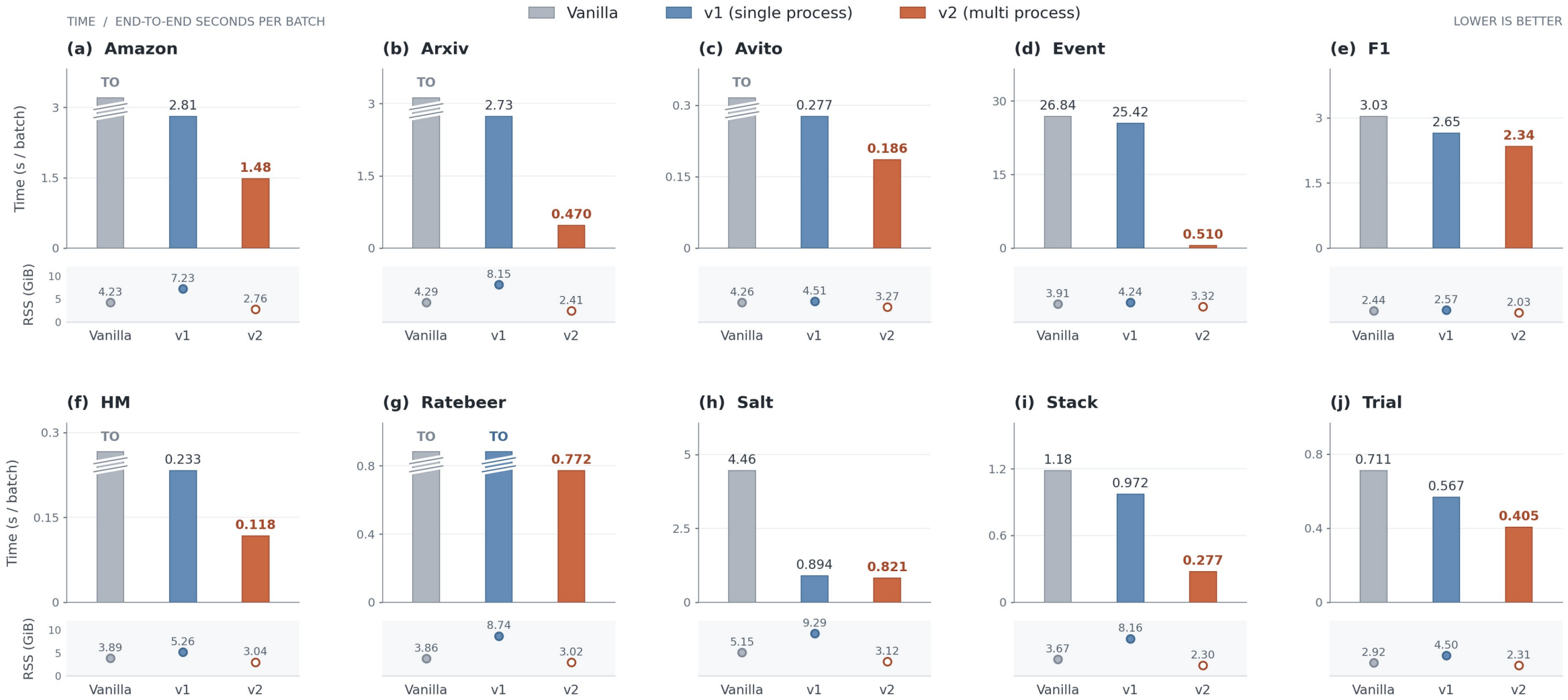
十数据集性能对比 | Vanilla · v1 · v2



按数据集类型解释（1/2）｜特征处理与数据供给

归类依据：数据特征 → 主导成本 → 对应策略；数据集只作为代表性证据。

文本编码重、可跨轮复用

例： Arxiv / Amazon / Stack

瓶颈：长文本与大词汇集合使重复编码昂贵， GPU 等待特征。

策略：直接文本编码＋并行组装；适合时持久化 embedding ，跨轮复用。

证据： Arxiv 两轮模型输入 184,724 → 1,316 ； 部分收益来自既有热缓存。

复用率高、缓存命中成本高

例： Salt / HM / Trial

瓶颈：减少编码后，共享查询、锁竞争和跨进程传输成为主要成本。

策略：本地槽位索引、批量取向量、只传请求行；必要时用 worker 本地缓存。

证据： HM ready 等待 3,348.9 → 481.9 秒；本地缓存为显式配置。

读取量大、特征供给仍重

例： Ratebeer / Avito

瓶颈： SQL 读取与组装成本高，未命中的绝对编码量仍可很大。

策略：采样 / 多进程组装 / GPU 训练重叠；缓存路径按实际复用成本选择。

边界： Ratebeer 第二轮仍编码约 2,050 万条；更快不等于消除供给瓶颈。

类别可以重叠；倍率是跨实验观察值，缓存状态不同； RSS 仅统计 v2 主进程。

按数据集类型解释（2/2）｜初始化与固定开销

归类依据：数据特征 → 主导成本 → 对应策略；数据集只作为代表性证据。

数值宽表、初始化占主导

例： Event

瓶颈：列多、逐列扫描与排序重复；少量训练无法摊薄初始化成本。

策略：按行数和内存预算分组扫描，精确 partition 分位数，复用初始化产物。

证据：冷初始化 2,164 → 606 秒（3.57×）；图中 49.83× 包含热初始化收益。

小数据集、固定开销占主导

例： F1

瓶颈：启动、进程协调和队列开销，相对实际训练工作量更突出。

策略：避免不必要的磁盘缓存；当前通用流水线收益有限，少 worker 是后续方向。

证据：图中 v1 / v2 仅 1.13×；不能预期小数据集获得与重文本任务同等收益。

优化原则：让策略匹配主导成本，而不是给所有数据集堆同一套缓存。

先减少重复工作，再降低命中成本；并行解决供给，预算控制初始化，固定开销限制小任务收益。

类别可以重叠；倍率是跨实验观察值，缓存状态不同； RSS 仅统计 v2 主进程。

Ratebeer ｜为何两个 TO ，只有 v2 有完成时间?

图中“完成”同时受吞吐、运行规模和预算影响，需要拆开解释。

Vanilla

4 小时超时

目标： 1 × 20,743 批
缺少可靠的完成批数记录

v1 (single process)

4 小时超时

完成： 15,686 / 20,743 批
观测吞吐约 0.918 秒 / batch

v2 (multi process)

4 小时 17 分完成

完成： 2 × 10,000 批（截断）
端到端 0.772 秒 / batch

性能：观测吞吐约快 1.19× ，单位 batch 耗时约少 15.9% 。

机制：并行采样、 SQL / 特征组装与 GPU 训练，改善供给；但 ready 等待仍达 11,389 秒，
两轮仍各有约 2,000 万条文本模型输入，说明 SQL 与编码瓶颈尚未消除。

稳定性：先将 TensorFrame 存储转为共享内存，再发布缓存条目，修复并发读取损坏；
修复后通过本次 20,000 批长跑。这是正确性证据，不能独立解释 1.19× 的性能收益。

结论： v2 在本次配置下更快且成功跑完；尚未证明在同一 4 小时预算完成完整两轮。